

VectorInject

SQL Injection Detection & Database Extraction Framework

Author

Abdulwahab Nashwan Ajlan

Course

Cybersecurity Practical Project

Supervisor

Dr. Ahmed Al-Suraihi

Version

1.0.0



The Problem

94%

of web applications have at least one vulnerability — making SQLi not a rare edge case, but the norm.

Why This Matters

→ OWASP Top 10 — #1 Threat

SQL Injection has consistently ranked as the most critical web application vulnerability for over a decade.

→ Manual Exploitation is Slow & Error-Prone

Crafting payloads by hand is tedious, inconsistent, and difficult to reproduce across targets.

→ Existing Tools Lack Transparency

Tools like SQLMap are powerful but opaque — students cannot easily learn *how* the detection logic works internally.

→ Educational Gap

There is a clear need for a transparent, readable, and educational SQLi framework built for learning.

What is **VectorInject**?

VectorInject is an advanced SQL Injection detection and exploitation framework designed from the ground up for educational clarity and practical effectiveness.



Pure Python

Zero external libraries — built entirely on the Python standard library for maximum transparency and portability.



Cross-Platform

Runs natively on both Kali Linux and Windows with no configuration changes required.



CLI Subcommands

Clean command-line interface with dedicated subcommands: `scan`, `detect`, `fingerprint`, `extract`.



Educational Purpose

Designed so students can read, trace, and understand exactly how each SQLi technique works at the code level.

Key Features

SQL Injection Detection

Supports four detection methods: **Error-based**, **Boolean-based**, **Time-based**, and **UNION-based** — covering the full spectrum of SQLi vulnerability classes.

Database Fingerprinting

Automatically identifies the target DBMS: **MySQL**, **PostgreSQL**, **MSSQL**, **Oracle**, and **SQLite** — adapting payloads accordingly.

Full Data Extraction

Hierarchical extraction of **Databases** → **Tables** → **Columns** → **Rows**, enabling complete enumeration of exposed data.

Session & Cookie Management

Automatic login handling via a built-in cookie jar — supports authenticated scanning without manual session injection.

Multi-threaded Scanning

Parallel vulnerability detection across parameters for significantly faster scan completion on large targets.

Result Export

Save all findings to structured output files for documentation, reporting, and further analysis.

Project Architecture

VectorInject follows a modular architecture with clear separation of concerns across eight core files.

Core Modules

01

`vectorinject.py`

Main CLI Entry Point — parses subcommands and routes execution to the appropriate module.

02

`core/scanner.py`

URL & Parameter Scanner — discovers injectable parameters across target endpoints.

03

`core/detector.py`

SQLi Detection Engine — applies all four detection techniques against discovered parameters.

04

`core/fingerprint.py`

Database Fingerprinting — identifies the backend DBMS using targeted probe payloads.

Support Modules

01

`core/extractor.py`

Data Extraction Engine — performs hierarchical enumeration of databases, tables, columns, and rows.

02

`utils/http_client.py`

HTTP Client with Cookie Jar — handles all network requests and session state management.

03

`utils/payloads.py`

SQL Injection Payloads — centralized payload library organized by technique and DBMS target.

04

`config/settings.py`

Global Configuration — timeouts, thread counts, output paths, and detection thresholds.

Detection Techniques

Error-Based

Triggers SQL syntax errors and detects them via **HTTP 500** status codes. Primary method used against DVWA.

Boolean-Based

Sends true/false condition payloads and compares response content to infer data without visible errors.

Time-Based

Injects `SLEEP()` delays and measures response time to detect blind SQLi where no output is returned.

UNION-Based

Appends `UNION SELECT` statements to extract data directly into the application's response output.

Example: Error-Based Detection Logic

```
payload = "1' OR '1'='1"
response = http_client.get(url, params={param: payload})
if response.status_code == 500:
    print("SQL Injection Detected!")
```

The detector sends a malformed payload that breaks SQL syntax. If the server returns HTTP 500, the parameter is flagged as injectable — a reliable signal when the application does not suppress error output.

Live Demo — DVWA Target

Target Configuration

URL: `http://localhost/dvwa/vulnerabilities/sqli/`

Parameter: `id`

Status: **VULNERABLE** — Error-based HTTP 500 confirmed

Extraction Results

Databases Extracted

`information_schema, dvwa`

Tables Extracted

`guestbook, users, security_log, access_log`

Users Extracted

`admin, gordonb, 1337, pablo, smithy`

Credentials

MD5 password hashes successfully recovered for all user accounts.



Detect

Fingerprint

Extract

Challenges Solved

Building VectorInject from scratch in pure Python required solving several non-trivial engineering problems.

Session Management

Implemented `http.cookiejar` to automatically handle login cookies — enabling authenticated scans against protected pages without manual session injection.

HTTP 500 Detection

DVWA suppresses visible SQL error messages, so the detector was adapted to identify injection by **HTTP status code** rather than response body content.

UNION-Based Row Extraction

Implemented row-by-row data retrieval using `LIMIT` and `OFFSET` clauses to paginate through result sets without knowing the total row count in advance.

Blank Page Detection

Identifies injection points by measuring **response size changes** — a blank or truncated response signals that the payload altered query execution.

Cross-Platform Compatibility

Handles Windows vs. Linux file path differences and line-ending conventions to ensure consistent behavior across both operating systems.

Conclusion & Future Work

Conclusion

→ Working SQLi Tool in Pure Python

Successfully built a fully functional detection and exploitation framework using only the Python standard library.

→ Real Vulnerability Detection & Exploitation

Detects and exploits live SQLi vulnerabilities across all four major technique classes.

→ Validated Against DVWA Lab

Extracted databases, tables, user accounts, and MD5 password hashes from a real DVWA instance.

Future Work



Blind SQLi Extraction

Add full data extraction support for time-based and boolean-based blind injection scenarios.



Automated POST-Form Injection

Extend scanning to automatically discover and test HTML form parameters via POST requests.



PDF Report Generation

Generate professional penetration testing reports automatically from scan results.



GUI Version

Build a graphical interface to make VectorInject accessible to beginners without CLI experience.

Thank You

Questions? I'm happy to walk through any part of the implementation, detection logic, or demo results.

Author

Abdulwahab Nashwan Ajlan

Project

VectorInject v1.0.0

Supervisor

Dr. Ahmed Al-Suraihi

GitHub

github.com/alwhababd447-dotcom/vectorinject

"Security through obscurity is not security. Understanding the attack is the first step to building the defense."